FinTechDevOps

Institutional Trading Terminal with End-to-End CI/CD & Kubernetes Delivery

Java Spring Boot, React, Docker, Kubernetes, Jenkins, Ansible,
ELK Stack, Ansible Vault, DockerHub and GitHub

CSE 816:

Software Production Engineering

Project Title	FinTechDevOps: Institutional Trading Terminal with End-to-End CI/CD & Kubernetes Delivery
Application Domain	Finance + DevOps + Cloud Native Infrastructure
Submitted By	Ashwin Suthar (MT2025024)
	Kakadiya Aman Jitendrabhai (MT2025058)
Professor	Dr. B Thangaraju
Institute	International Institute of Information Technology Bangalore
Academic Year	2025–2026

Department of Computer Science and Engineering
International Institute of Information Technology Bangalore

May 2026

ABSTRACT

FinTechDevOps is a cloud-native institutional trading terminal built with a microservices architecture using Java Spring Boot, React, and Python. The project demonstrates modern Software Production Engineering methodologies by automating the complete Software Development Life Cycle (SDLC) through an integrated DevOps pipeline.

The system comprises nine Java Spring Boot microservices (Config Server, Eureka Service Discovery, API Gateway, User Service, Order Service, Portfolio Service, Market Data Service, Notification Service, and Risk Service), a Python-based AI price prediction engine, and a React frontend dashboard—all containerized with Docker and orchestrated via Kubernetes.

The CI/CD pipeline is implemented using Jenkins with GitHub Webhook triggers. On every commit, Jenkins automatically checks out code, runs lint checks and unit tests, builds Docker images, performs container security scans, pushes images to DockerHub, and deploys to Kubernetes **through Ansible Playbooks**—not directly from Jenkins. Ansible Vault encrypts all sensitive credentials, and Ansible Roles provide modular infrastructure management.

Kubernetes Horizontal Pod Autoscaling (HPA) enables dynamic scaling based on CPU utilization. The ELK Stack (Elasticsearch, Logstash, Kibana) provides centralized logging and monitoring with all microservice logs feeding into Logstash via the `logstash-logback-encoder`. HashiCorp Vault manages runtime secrets, and Kubernetes Secrets handle deployment-time credentials.

The platform delivers real-time stock trading with buy/sell execution, portfolio holdings tracking, transaction audit logging, and an AI-powered price prediction module—all deployed as a scalable, secure, production-grade Finance-domain DevOps ecosystem.

Contents

1 Project Overview **5**
- 1.1 Introduction . . . 5
- 1.2 Problem Statement . . . 5
- 1.3 Solution Implemented . . . 6

2 Project Architecture **7**
- 2.1 Architecture Overview . . . 7
- 2.2 Microservices Architecture . . . 9
- 2.3 Infrastructure Layer . . . 9

3 Technology Stack **11**
- 3.1 Technology Selection and Justification . . . 11

4 Frontend Design and User Interface **13**
- 4.1 Home Page . . . 13
- 4.2 Trading Terminal . . . 14
- 4.3 Portfolio Holdings Page . . . 15
- 4.4 Audit Log Page . . . 16
- 4.5 Login Page . . . 16

5 CI/CD Pipeline and Deployment Automation **17**
- 5.1 Jenkins Pipeline Architecture . . . 17
- 5.2 Key Design Decision: Jenkins → Ansible → Kubernetes . . . 18

6 Infrastructure Automation using Ansible **19**
- 6.1 Ansible Playbooks and Roles . . . 19
- 6.2 Ansible Vault Integration . . . 19
- 6.3 Deployment Execution Flow . . . 20

7 Kubernetes Orchestration and Scalability **21**
- 7.1 Kubernetes Deployment Architecture . . . 21
- 7.2 Horizontal Pod Autoscaling (HPA) . . . 21

8 Centralized Logging and Monitoring **23**
- 8.1 ELK Stack Integration . . . 23

9 Security and DevSecOps Features **24**
9.1 Multi-Layer Security Architecture 24

10 Innovative Features **25**

11 Results and Observations **26**
11.1 Deployment Results 26

12 Conclusion and Team Contributions **27**
12.1 Conclusion 27
12.2 Individual Team Contributions 27

13 Future Scope **29**

References **30**

List of Figures

2.1 Overall Cloud-Native System Architecture 8

4.1 Home Page — Main Trading Dashboard Overview 13
4.2 Trading Terminal — Order Execution Interface with BUY/SELL Controls 14
4.3 Portfolio Holdings Page — Real-Time Asset Inventory with Average Buy Prices 15
4.4 Audit Log Page — Complete Trade Execution History with Timestamps 16
4.5 Login Page — Secure Institutional Authentication Interface 16

5.1 Jenkins CI/CD Pipeline Stage View with Execution Times 18

Chapter 1

Project Overview

1.1 Introduction

Modern financial software demands automated deployment, scalable infrastructure, centralized monitoring, and secure runtime environments. Traditional manual deployment approaches fail to meet the reliability and speed requirements of institutional trading systems.

FinTechDevOps was developed as a Finance-domain Software Production Engineering project that combines a fully functional trading platform with enterprise-grade DevOps automation. The system provides:

- Real-time stock monitoring and trading execution
- Portfolio holdings management with average buy price tracking
- Comprehensive transaction audit logging with timestamps
- AI-powered stock price prediction using Python
- Fully automated CI/CD pipeline triggered on every Git commit
- Kubernetes-based scalable deployment orchestrated through Ansible
- Centralized logging via the ELK Stack

1.2 Problem Statement

Traditional software deployment in financial environments suffers from manual server configuration, inconsistent environments, slow delivery cycles, poor scalability, and lack of centralized monitoring. These limitations are critical in Finance-domain applications where high availability, secure infrastructure, audit traceability, and minimal downtime are mandatory.

How can a Finance-domain application be deployed using fully automated cloud-native DevOps workflows while ensuring security, scalability, observability, and production-grade infrastructure reliability?

The proposed solution combines Jenkins CI/CD automation, Docker containerization, Kubernetes orchestration, Ansible-based Infrastructure as Code, Ansible Vault for secure credential management, Kubernetes HPA for dynamic scaling, and ELK Stack for centralized monitoring.

1.3 Solution Implemented

The FinTechDevOps platform implements a complete cloud-native deployment ecosystem where:

1. Developers push code to the GitHub repository
2. GitHub Webhooks trigger the Jenkins pipeline automatically
3. Jenkins performs lint checks, unit tests, Docker builds, security scans, and DockerHub pushes
4. Jenkins invokes **Ansible Playbooks** (not raw `kubectl` commands)
5. Ansible decrypts Vault-encrypted secrets and applies Kubernetes manifests
6. Kubernetes deploys 19 containerized workloads with HPA-enabled scaling
7. The ELK Stack aggregates and visualizes all runtime logs in Kibana
8. Email notifications report build success/failure to the team

This architecture satisfies every requirement of the CSE 816 project guidelines while demonstrating real-world enterprise DevOps practices within the Finance domain.

Chapter 2

Project Architecture

2.1 Architecture Overview

The platform follows a microservices architecture with nine Java Spring Boot services registered through Eureka Service Discovery, fronted by a Spring Cloud API Gateway, and backed by PostgreSQL, Redis, and Apache Kafka.

The complete deployment flow is:

GitHub → Jenkins → Docker Build → DockerHub → Ansible Playbooks → Kubernetes Cluster → ELK Monitoring

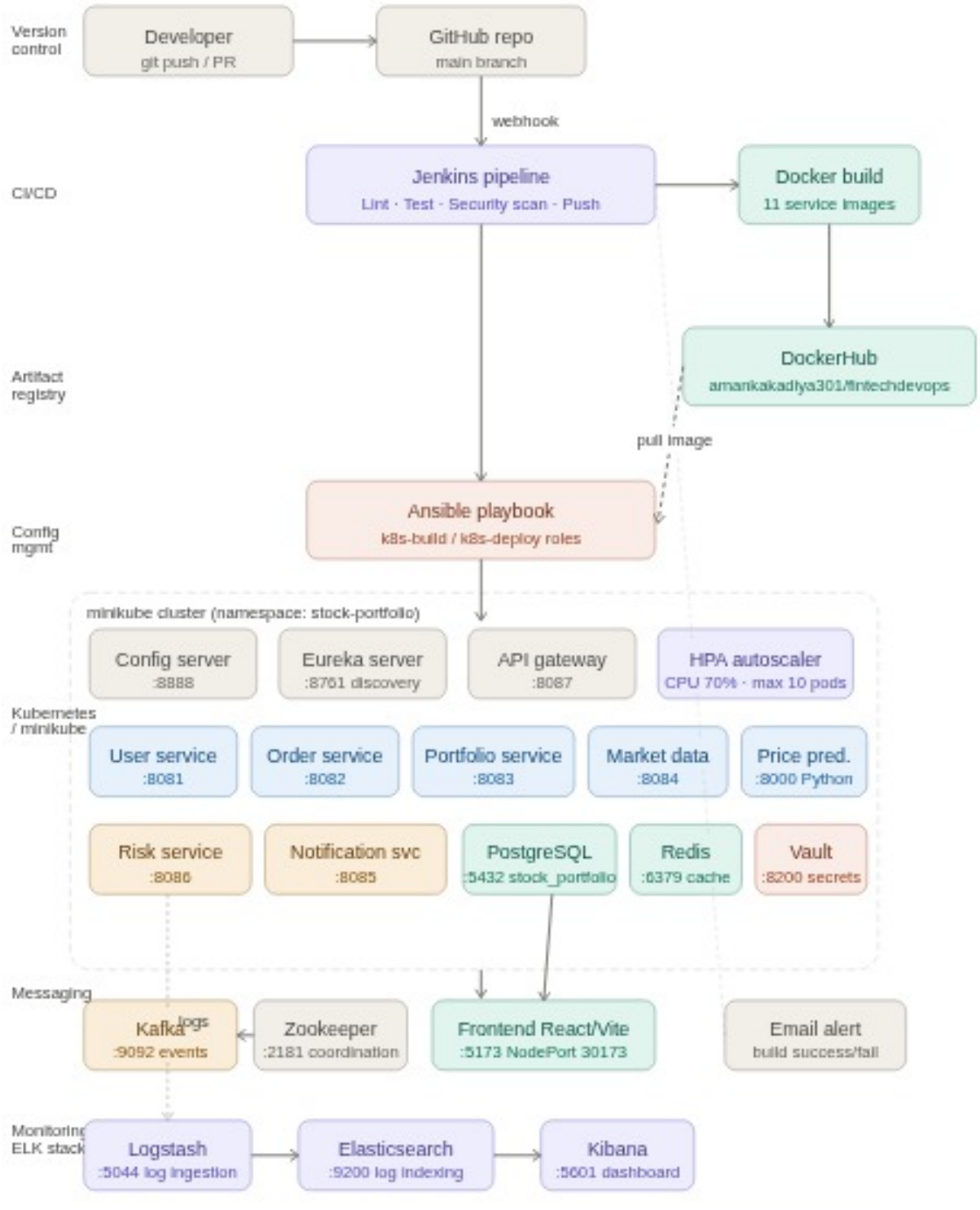


Figure 2.1: Overall Cloud-Native System Architecture

2.2 Microservices Architecture

Each microservice is independently deployable and communicates through REST APIs via the API Gateway.

Service	**Port**	**Responsibility**
Config Server	8888	Centralized configuration management
Eureka Server	8761	Service discovery and registration
API Gateway	8080	Request routing, CORS, and load balancing
User Service	8081	Authentication, registration, JWT tokens
Order Service	8082	Trade execution (buy/sell) with validation
Portfolio Service	8083	Holdings management and balance tracking
Market Data Service	8084	Real-time stock price feeds
Notification Service	8085	Alert and notification delivery
Risk Service	8086	Trade risk assessment and validation
Price Prediction (Python)	8000	AI-based stock price prediction engine
Frontend (React)	5174	Institutional trading dashboard UI

Table 2.1: Microservices and Their Responsibilities

2.3 Infrastructure Layer

The infrastructure backbone runs as Docker containers and Kubernetes pods:

Component	Purpose
PostgreSQL	Primary relational database for users, orders, and portfolios
Redis	In-memory caching for session and market data
Apache Kafka + Zookeeper	Event streaming for inter-service communication
HashiCorp Vault	Runtime secret management and credential storage
Elasticsearch	Log indexing and full-text search
Logstash	Log ingestion pipeline from microservices
Kibana	Log visualization and monitoring dashboards

Table 2.2: Infrastructure Components

Chapter 3

Technology Stack

3.1 Technology Selection and Justification

Each technology was selected based on its suitability for enterprise-grade Finance-domain deployment:

Technology	**Category**	**Why It Was Used**
Java Spring Boot	Backend	Industry-standard for microservices with built-in dependency injection, security, and JPA support
React + Vite	Frontend	Component-based UI with fast hot-reload for rapid development
Python (FastAPI)	AI/ML	Lightweight framework for the price prediction engine
PostgreSQL	Database	ACID-compliant relational database essential for financial transactions
Docker	Containerization	Ensures consistent environments from development to production
Kubernetes	Orchestration	Production-grade container orchestration with self-healing and scaling
Jenkins	CI/CD	Mature pipeline automation with extensive plugin ecosystem
Ansible	IaC	Agentless infrastructure automation for deploying K8s manifests
Ansible Vault	Security	Encrypts sensitive variables (DB passwords, tokens) at rest
Ansible Roles	Modularity	Separates deployment logic into reusable `k8s-build` and `k8s-deploy` roles
K8s HPA	Scalability	Auto-scales pods based on CPU utilization (70% threshold, max 10 replicas)
ELK Stack	Monitoring	Centralized log aggregation, processing, and visualization
GitHub	Version Control	Source code management with webhook-triggered CI/CD
DockerHub	Registry	Container image storage and distribution

Table 3.1: Complete Technology Stack with Justification

Chapter 4

Frontend Design and User Interface

The platform provides five core interfaces for institutional trading operations. Each screenshot below represents a distinct functional module of the deployed application.

4.1 Home Page

The home page is the main landing dashboard after login. It presents an overview of the operator's trading environment including the real-time cash balance, active stock positions, and quick-access navigation to holdings and audit logs. The interface follows an institutional dark-theme design inspired by professional Bloomberg and Reuters terminals.

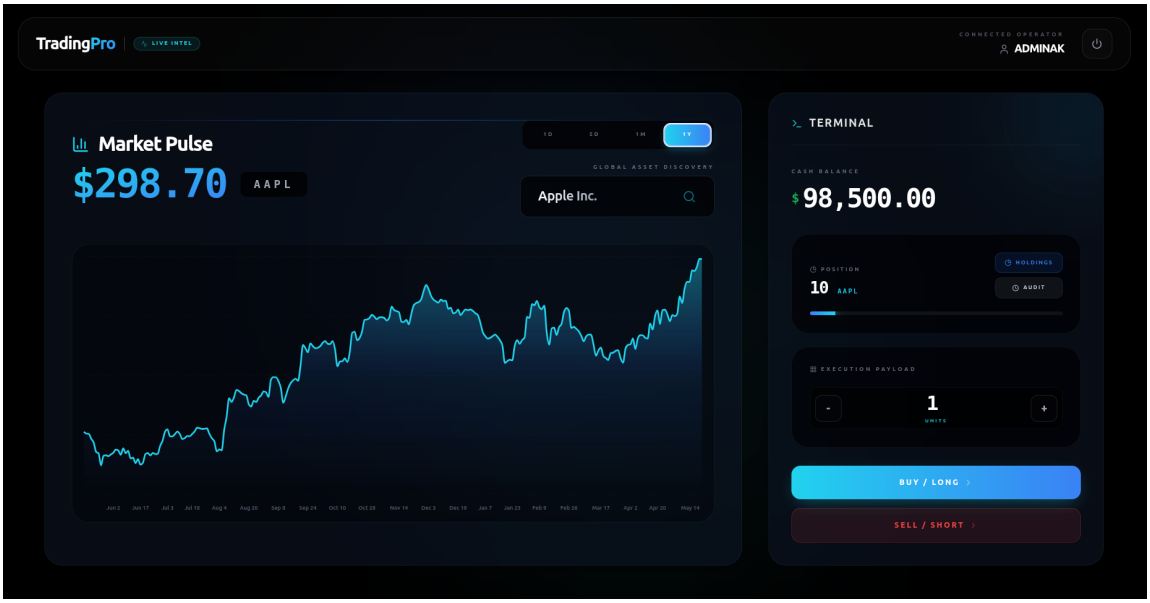


Figure 4.1: Home Page — Main Trading Dashboard Overview

4.2 Trading Terminal

The trading terminal is the order execution interface where operators submit BUY/LONG and SELL/SHORT orders. It displays the current cash balance (e.g., $96,476.00), the active position with ticker symbol and quantity, and an execution payload panel for selecting the number of units. Orders are validated by the Order Service through the Spring Cloud API Gateway before execution.

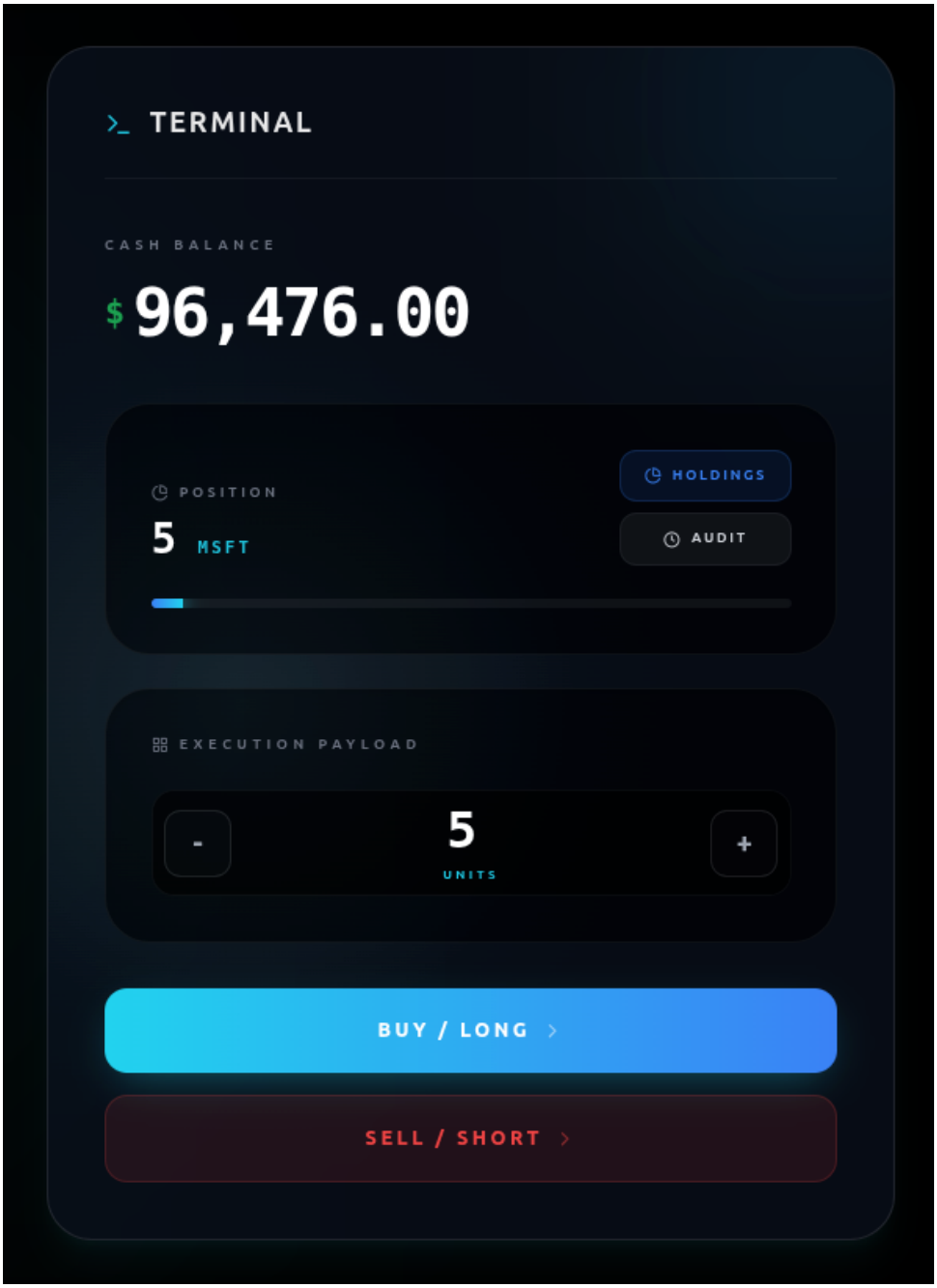


Figure 4.2: Trading Terminal — Order Execution Interface with BUY/SELL Controls

4.3 Portfolio Holdings Page

The holdings page displays all currently held stock positions. Each card shows the asset ticker (e.g., MSFT, AAPL), net units owned, and the dynamically computed average buy price. This data is fetched from the Portfolio Service, which maintains a real-time inventory synchronized with the Order Service.

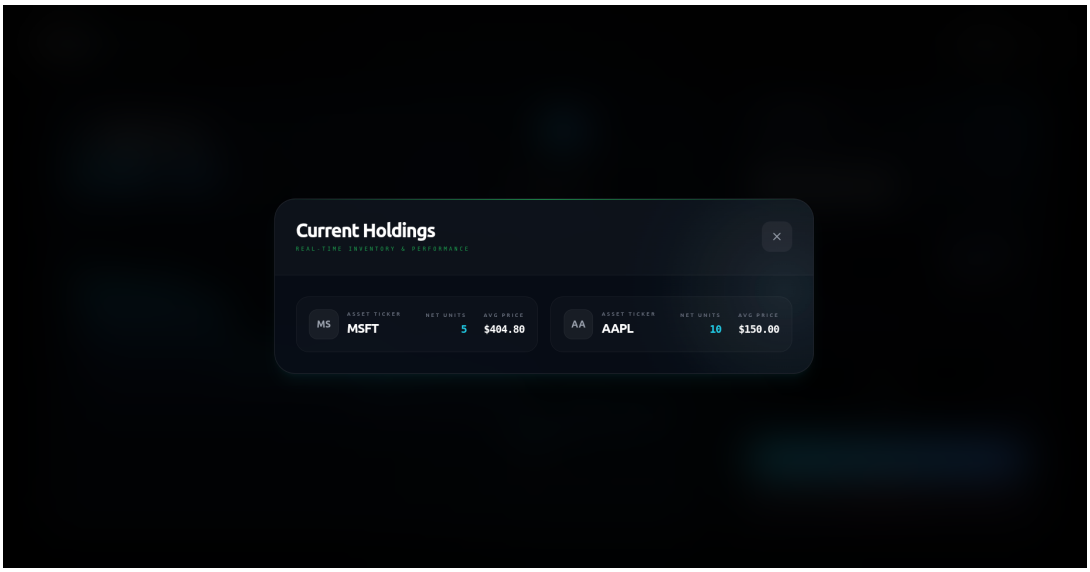


Figure 4.3: Portfolio Holdings Page — Real-Time Asset Inventory with Average Buy Prices

4.4 Audit Log Page

The audit log page provides a comprehensive trade execution history. Every transaction is recorded with a timestamp, ticker symbol, action type (BUY or SELL), quantity, settlement price, and total value. This immutable audit trail is essential for financial compliance, operational transparency, and debugging.

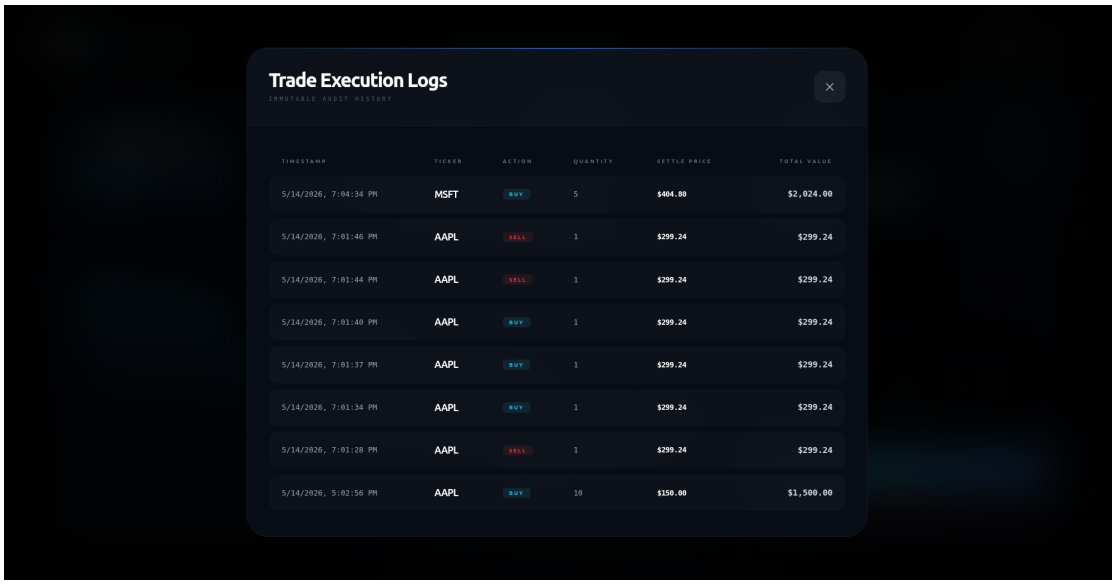


Figure 4.4: Audit Log Page — Complete Trade Execution History with Timestamps

4.5 Login Page

The login page provides secure operator authentication using JWT-based tokens. Users enter their credentials (Operator Identity and Security Key), which are validated against the User Service backend. Upon successful authentication, a JWT token is issued for session management across all protected API endpoints.

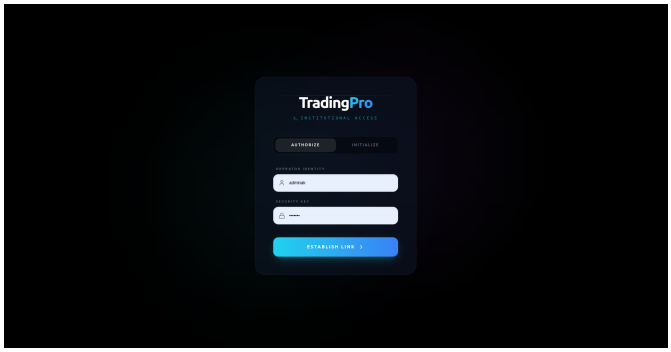


Figure 4.5: Login Page — Secure Institutional Authentication Interface

Chapter 5

CI/CD Pipeline and Deployment Automation

5.1 Jenkins Pipeline Architecture

The Jenkins pipeline automates the entire SDLC. It is triggered automatically via GitHub Webhooks on every commit to the `main` branch using the `githubPush()` trigger. The pipeline runs on port 9090 and consists of the following stages:

1. **Checkout**: Fetches the latest code from the GitHub repository
2. **Lint**: Compiles all nine Java microservices using Gradle to validate code integrity
3. **Lint & Secrets Check**: Scans the codebase for hardcoded credentials and API keys
4. **Unit Tests**: Executes JUnit test suites across all microservices
5. **Build Docker Image**: Builds Docker images for all 11 services (9 Java + 1 Python + 1 React)
6. **Container Security Scan**: Runs Docker Scout to identify critical and high-severity CVEs
7. **Push to Registry**: Pushes all images to DockerHub (`amankakadiya301/fintechdevops`)
8. **Deploy to Kubernetes**: Invokes `ansible-playbook ansible/playbook.yml` — **not** raw `kubectl` commands
9. **Post-Deploy Smoke Test**: Validates health endpoints of Eureka, API Gateway, and User Service
10. **Email Notification**: Sends build status (SUCCESS/FAILURE) to `kakadiyaaman2004@gmail.co`

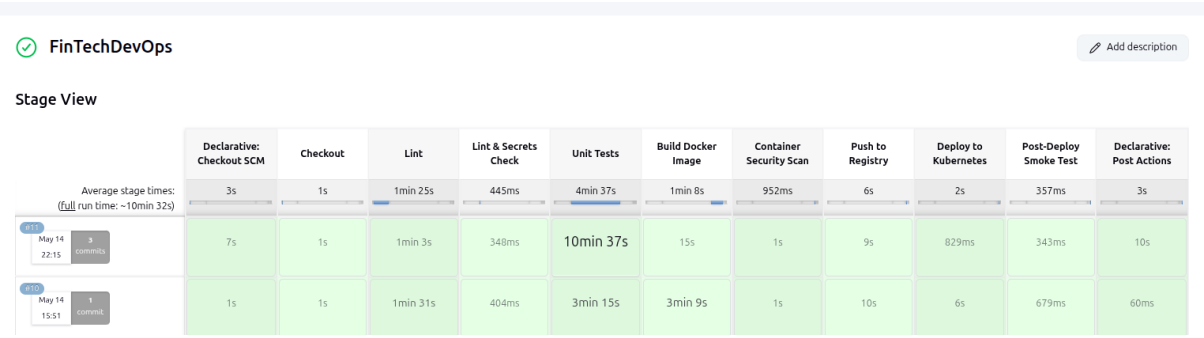


Figure 5.1: Jenkins CI/CD Pipeline Stage View with Execution Times

5.2 Key Design Decision: Jenkins → Ansible → Kubernetes

A critical architectural requirement was that Kubernetes deployment manifests must **not** be executed directly from Jenkins. Instead, Jenkins calls Ansible Playbooks, which in turn apply the Kubernetes manifests. This three-tier deployment strategy provides:

- **Modularity**: Ansible Roles separate build and deploy concerns
- **Security**: Ansible Vault decrypts secrets before deployment
- **Reproducibility**: Infrastructure as Code ensures consistent deployments
- **Auditability**: Ansible provides detailed execution logs for every deployment step

Chapter 6

Infrastructure Automation using Ansible

6.1 Ansible Playbooks and Roles

The Ansible deployment architecture uses modular roles to separate infrastructure concerns:

Component	**Purpose**
`playbook.yml`	Master playbook that orchestrates the complete K8s deployment
`playbook-build.yml`	Manages Docker image building via Kubernetes jobs
Role: `k8s-deploy`	Creates namespace, applies Vault secrets, deploys all services, enables HPA
Role: `k8s-build`	Manages Docker image builder jobs within Kubernetes
`vars/secrets.yml`	Ansible Vault-encrypted file containing DB passwords and Docker credentials

Table 6.1: Ansible Infrastructure Components

6.2 Ansible Vault Integration

Sensitive deployment credentials are encrypted using Ansible Vault. The `vars/secrets.yml` file contains database passwords, Docker registry tokens, and API keys—all encrypted at rest. During deployment, Ansible decrypts these variables using a secure vault password file before injecting them into Kubernetes Secrets.

This architecture ensures that no plaintext credentials are stored in the repository,

satisfying DevSecOps best practices and the project's security requirements.

6.3 Deployment Execution Flow

The `k8s-deploy` role executes four sequential tasks:

1. Ensures the `stock-portfolio` Kubernetes namespace exists
2. Applies Vault configuration (`vault-secret.yaml`) for Kubernetes Secrets
3. Deploys all 19 microservice and infrastructure workloads (`deployment.yaml`)
4. Applies the Horizontal Pod Autoscaler configuration (`hpa.yaml`)

All Kubernetes manifests are referenced using `playbook_dir` relative paths, ensuring portability across different deployment environments.

Chapter 7

Kubernetes Orchestration and Scalability

7.1 Kubernetes Deployment Architecture

The entire platform runs within a Kubernetes cluster (Minikube) under the `stock-portfolio` namespace. All 19 workloads are defined in a single declarative manifest (`deployment.yaml`) that specifies:

- Deployment specifications with resource limits (CPU and memory)
- Service definitions for inter-pod communication
- Environment variable injection from Kubernetes Secrets (Vault keys)
- Container image references pointing to DockerHub
- Rolling update strategies for zero-downtime deployments

The 19 deployed pods include: Zookeeper, Kafka, PostgreSQL, Redis, HashiCorp Vault, Elasticsearch, Kibana, Logstash, Config Server, Eureka Server, API Gateway, User Service, Order Service, Portfolio Service, Market Data, Notification Service, Risk Service, Price Prediction, and Frontend React.

7.2 Horizontal Pod Autoscaling (HPA)

Kubernetes HPA dynamically scales the Order Execution Service based on runtime CPU utilization:

HPA Parameter	**Configuration**
Target Deployment	`order-execution-service`
Minimum Replicas	1 Pod
Maximum Replicas	10 Pods
Target CPU Utilization	70%
API Version	`autoscaling/v2`

Table 7.1: Horizontal Pod Autoscaler Configuration

When CPU utilization exceeds 70%, Kubernetes automatically provisions additional pods. During low traffic, excess replicas are terminated to optimize resource consumption. This ensures the trading engine remains responsive during high-volume trading periods.

Chapter 8

Centralized Logging and Monitoring

8.1 ELK Stack Integration

The platform integrates the ELK Stack for centralized log aggregation, processing, and visualization:

- **Logstash**: Receives structured JSON logs from all Java microservices on port 5044 via TCP. A custom Logstash pipeline filters and tags trading-related logs for specialized monitoring.
- **Elasticsearch**: Indexes all ingested logs into daily indices (`microservices-logs-YYYY.MM.dd`) for efficient querying and retention management.
- **Kibana**: Provides web-based dashboards for real-time log visualization, allowing operators to monitor trading activity, authentication events, and infrastructure health.

Every Java microservice includes the `logstash-logback-encoder` dependency and a custom `logback-spring.xml` configuration that streams structured JSON logs to the Logstash endpoint. The `LOGSTASH_HOST` environment variable is injected into each Kubernetes pod to enable automatic log routing.

Chapter 9

Security and DevSecOps Features

9.1 Multi-Layer Security Architecture

The platform implements security at every layer of the deployment stack:

Layer	**Technology**	**Protection**
Application	JWT Tokens	Stateless authentication for API access
API Gateway	Spring Security	CORS enforcement and request filtering
Infrastructure	Ansible Vault	Encrypted credentials in repository
Runtime	K8s Secrets	Secure environment variable injection
Secrets Mgmt	HashiCorp Vault	Centralized runtime secret storage
CI/CD	Docker Scout	Container vulnerability scanning
Pipeline	Secrets Scanner	Detects hardcoded credentials in code

Table 9.1: Multi-Layer Security Implementation

Chapter 10

Innovative Features

The following innovative solutions differentiate this project from conventional academic implementations:

1. **AI-Powered Price Prediction**: A Python FastAPI microservice uses machine learning models to provide real-time stock price predictions, integrating AIOps with traditional DevOps workflows.

2. **Institutional-Grade Trading UI**: The frontend implements a premium glassmorphic design with dark theme, volumetric lighting effects, and micro-animations—inspired by real Bloomberg and Reuters trading terminals.

3. **Financial Validation Engine**: The Order Service enforces strict balance checks (rejecting BUY orders exceeding available cash) and inventory tracking (preventing SELL orders exceeding held shares), simulating real brokerage risk management.

4. **Three-Tier Deployment Architecture**: The Jenkins → Ansible → Kubernetes deployment chain mirrors enterprise infrastructure workflows, providing modularity and security separation that direct Jenkins-to-K8s deployments cannot achieve.

5. **Automated Email Notifications**: Build status reports are automatically emailed to team members using Gmail SMTP integration with the Jenkins Extended Email plugin, ensuring immediate visibility of pipeline health.

6. **Container Security Scanning**: Docker Scout integration within the CI/CD pipeline automatically identifies critical and high-severity CVEs before images are pushed to the registry, implementing shift-left security practices.

7. **Dynamic Average Buy Price**: The portfolio system dynamically computes weighted average buy prices across multiple purchase transactions, providing accurate cost-basis tracking for each held asset.

Chapter 11

Results and Observations

11.1 Deployment Results

The platform was successfully deployed with all components fully operational:

Requirement	**Status**
Git + GitHub Version Control	Successfully Implemented
Jenkins CI/CD with GitHub Webhooks	Successfully Implemented
Docker Containerization	Successfully Implemented (11 images)
Ansible Playbooks for K8s Deployment	Successfully Implemented
Ansible Vault Credential Encryption	Successfully Implemented
Ansible Roles (k8s-build, k8s-deploy)	Successfully Implemented
Kubernetes Orchestration	Successfully Implemented (19 pods)
Kubernetes HPA	Successfully Implemented (70% CPU, max 10)
ELK Stack Monitoring	Successfully Implemented
Email Build Notifications	Successfully Implemented
Finance Domain Application	Successfully Implemented
AI Price Prediction (Python)	Successfully Implemented

Table 11.1: Final Project Implementation Results

Chapter 12

Conclusion and Team Contributions

12.1 Conclusion

The FinTechDevOps platform successfully demonstrates practical implementation of Software Production Engineering methodologies through a fully automated cloud-native DevOps ecosystem. The deployment architecture follows the mandated flow—GitHub → Jenkins → Ansible Playbooks → Kubernetes—ensuring Infrastructure as Code principles, secure credential management, modular deployment orchestration, and dynamic scalability.

The project bridges the gap between academic learning and enterprise-grade Software Production Engineering by implementing a Finance-domain trading system with real-world deployment automation, security practices, and operational monitoring.

12.2 Individual Team Contributions

Ashwin Suthar (MT2025024)

- Java Spring Boot microservices development (User Service, Order Service, Portfolio Service)
- Docker containerization of backend microservices and frontend application
- Kubernetes deployment manifest authoring (`deployment.yaml` for 19 workloads)
- Kubernetes HPA configuration for dynamic pod autoscaling
- ELK Stack integration: Logstash pipeline configuration and Kibana dashboard setup
- Frontend React trading dashboard, holdings visualization, and audit log interface
- API Gateway CORS configuration and Spring Cloud routing setup
- System architecture design and project documentation

Kakadiya Aman Jitendrabhai (MT2025058)

- Jenkins CI/CD pipeline implementation with 10-stage automated workflow
- GitHub Webhook integration and ngrok tunnel configuration for trigger automation
- Ansible Playbook and Role implementation (`k8s-build`, `k8s-deploy`) for IaC
- Ansible Vault setup for encrypted credential management (`secrets.yml`)
- Docker image lifecycle management and DockerHub registry push automation
- Kubernetes cluster orchestration, namespace management, and pod deployment
- ELK Stack integration: `logstash-logback-encoder` injection across all 9 Java services
- Container security scanning using Docker Scout in CI/CD pipeline
- Email notification integration (SMTP) for build status alerts
- Infrastructure debugging, smoke testing, and deployment validation

Collaborative Contributions

- Cloud-native microservices architecture planning and technology selection
- End-to-end deployment testing across Docker, Kubernetes, and Ansible layers
- Runtime validation, integration verification, and pipeline troubleshooting
- Report preparation and project documentation

Chapter 13

Future Scope

- Integration with live stock market APIs for real-time data feeds
- WebSocket-based sub-millisecond price updates replacing REST polling
- Multi-node Kubernetes cluster deployment on AWS EKS or Azure AKS
- GitOps-based deployment using ArgoCD for declarative infrastructure
- Service mesh integration using Istio for advanced traffic management
- Distributed database replication with PostgreSQL streaming replication
- Advanced RBAC security policies with Kubernetes NetworkPolicies
- Production-grade disaster recovery and backup automation

References

1. Docker Documentation, *Docker Container Platform*.
   Available at: https://docs.docker.com/

2. Kubernetes Documentation, *Production-Grade Container Orchestration*.
   Available at: https://kubernetes.io/docs/

3. Jenkins Documentation, *CI/CD Automation Server*.
   Available at: https://www.jenkins.io/doc/

4. Ansible Documentation, *Infrastructure Automation*.
   Available at: https://docs.ansible.com/

5. Elastic Stack Documentation, *Elasticsearch, Logstash and Kibana*.
   Available at: https://www.elastic.co/docs

6. Spring Boot Documentation, *Java Microservices Framework*.
   Available at: https://spring.io/projects/spring-boot

7. React Documentation, *Frontend UI Library*.
   Available at: https://react.dev/

8. GitHub Documentation, *Version Control and Repository Management*.
   Available at: https://docs.github.com/

9. Kubernetes HPA Documentation, *Horizontal Pod Autoscaler*.
   Available at: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autos

10. HashiCorp Vault Documentation, *Secrets Management*.
    Available at: https://developer.hashicorp.com/vault/docs